

Contents

Opérations mathématiques en Python	3
1. Nature des nombres en Python	3
2. Addition	3
Point important	4
3. Soustraction	4
4. Multiplication	4
5. Division réelle (/)	4
Piège fréquent	4
6. Division entière (//)	5
Cas négatif (important)	5
7. Modulo (%)	5
8. Puissance (**)	5
9. Priorité des opérations	6
10. Mélange de types	6
11. Conversion explicite	6
12. Erreurs classiques	7
1) Division par zéro	7
2) Mélange chaîne / nombre	7
13. Précision des float (concept fondamental)	7
14. Cas métier simple : calcul de facture	8
Question de réflexion	8
15. Bonnes pratiques modernes	8
16. Mini-exercices de prédiction	8
17. Synthèse	9
20 Questions intelligentes — Opérations mathématiques en Python	9
1) Pourquoi $0.1 + 0.2$ ne donne pas exactement 0.3 ?	9
Énoncé	9
Solution	9

2) Pourquoi $4 / 2$ retourne 2.0 et non 2 ?	10
Énoncé	10
Solution	10
3) Quelle différence entre $//$ et $\text{int}(a / b)$?	10
Énoncé	10
Solution	10
4) Pourquoi $-5 // 2$ retourne -3 ?	11
Énoncé	11
Solution	11
5) Relation fondamentale entre $//$ et $\%$	11
Énoncé	11
Solution	11
6) Pourquoi $\text{int}(3.9)$ ne donne pas 4 ?	12
Énoncé	12
Solution	12
7) Pourquoi $\text{round}(2.5)$ donne 2 ?	12
Énoncé	12
Solution	12
8) Pourquoi $10 \% -3$ donne -2 ?	13
Énoncé	13
Solution	13
9) Priorité des opérateurs	13
Énoncé	13
Solution	13
10) Pourquoi $1e3$ est valide ?	13
Énoncé	13
Solution	13
11) Pourquoi <code>bool</code> est un <code>int</code> ?	14
Énoncé	14
Solution	14
12) Pourquoi $2 ** 1000$ fonctionne ?	14
Énoncé	14
Solution	14
13) Pourquoi $0.3 == 0.1 + 0.2$ est faux ?	15
Énoncé	15
Solution	15
14) Différence entre $+=$ et $+$?	15
Énoncé	15
Solution	15
15) Pourquoi <code>float("3.5")</code> fonctionne mais <code>float("3,5")</code> non ?	15
Énoncé	15
Solution	16
16) Différence entre <code>abs()</code> et <code>math.fabs()</code>	16
Énoncé	16
Solution	16
17) Pourquoi <code>pow()</code> existe alors que <code>**</code> existe ?	16

Énoncé	16
Solution	16
18) Division avec nombres très grands	17
Énoncé	17
Solution	17
19) Pourquoi $5 * "3"$ fonctionne mais $5 + "3"$ non ?	17
Énoncé	17
Solution	17
20) Cas métier : calcul de remise	18
Énoncé	18
Solution	18

Opérations mathématiques en Python

Le support est une propriété de **Bechir Bejaoui**

1. Nature des nombres en Python

Python manipule principalement trois types numériques :

- `int` → nombres entiers
- `float` → nombres décimaux (représentation binaire IEEE 754)
- `complex` → nombres complexes (hors périmètre ici)

Vérification rapide :

```
print(type(10))      # int
print(type(10.5))    # float
```

2. Addition

L'opérateur `+` additionne deux valeurs numériques.

```
print(2 + 3)
print(2 + 3.0)
```

Sortie attendue :

```
5
5.0
```

Point important

Si un float est impliqué → le résultat devient float.

3. Soustraction

```
print(10 - 4)
print(10.5 - 4)
```

Sortie :

```
6
6.5
```

4. Multiplication

```
print(4 * 5)
print(4 * 2.5)
```

Sortie :

```
20
10.0
```

5. Division réelle (/)

Toujours un float, même si le résultat est exact.

```
print(10 / 2)
print(10 / 4)
```

Sortie :

```
5.0
2.5
```

Piège fréquent

```
print(4 / 2)
```

Sortie :

```
2.0    # pas 2
```

6. Division entière (//)

Renvoie la partie entière après division.

```
print(10 // 3)
print(10 // 2)
```

Sortie :

```
3
5
```

Cas négatif (important)

```
print(-10 // 3)
```

Sortie :

```
-4
```

Explication :

La division entière tronque vers le bas (floor), pas vers zéro.

7. Modulo (%)

Renvoie le reste de la division.

```
print(10 % 3)
```

Sortie :

```
1
```

Relation mathématique :

$$a == (a // b) * b + (a \% b)$$

Vérification :

```
a = 10
b = 3
```

```
print(a == (a // b) * b + (a % b))
```

8. Puissance (**)

```
print(2 ** 3)
print(9 ** 0.5)
```

Sortie :

8
3.0

9. Priorité des opérations

Ordre :

1. Parenthèses
2. Puissance
3. Multiplication / Division / Division entière / Modulo
4. Addition / Soustraction

Exemple :

```
print(3 + 4 * 2)
print((3 + 4) * 2)
```

Sortie :

11
14

10. Mélange de types

```
print(type(3 + 2.0))
```

Sortie :

```
<class 'float'>
```

11. Conversion explicite

```
print(int(3.9))
print(float(3))
```

Sortie :

3
3.0

Attention :
int() tronque, n'arrondit pas.

12. Erreurs classiques

1) Division par zéro

```
print(5 / 0)
```

Erreur :

ZeroDivisionError

2) Mélange chaîne / nombre

```
print("2" + 2)
```

Erreur :

TypeError

Solution :

```
print(int("2") + 2)
```

13. Précision des float (concept fondamental)

```
print(0.1 + 0.2)
```

Sortie :

0.30000000000000004

Explication :

Représentation binaire approximative des nombres décimaux.

Solution métier (finance, facturation) :

```
from decimal import Decimal
```

```
a = Decimal("0.1")
```

```
b = Decimal("0.2")
```

```
print(a + b)
```

Sortie :

0.3

14. Cas métier simple : calcul de facture

```
prix_unitaire = 19.99
quantite = 3
tva = 0.19

total_ht = prix_unitaire * quantite
total_ttc = total_ht * (1 + tva)

print("Total HT:", total_ht)
print("Total TTC:", total_ttc)
```

Question de réflexion

Pourquoi ce calcul peut être dangereux en production financière ?

15. Bonnes pratiques modernes

- Ne jamais utiliser float pour de la finance
 - Toujours contrôler les divisions
 - Utiliser des parenthèses pour éviter toute ambiguïté
 - Comprendre la différence entre `//` et `/`
 - Tester les cas négatifs
 - Éviter les conversions implicites floues
-

16. Mini-exercices de prédiction

Prédire la sortie avant exécution :

```
print(5 // 2)
print(5 / 2)
print(5 % 2)
print(-5 // 2)
```

17. Synthèse

Opérateur	Rôle	Type de retour
-----------	------	----------------

- Addition | int ou float
- Soustraction | int ou float
- Multiplication | int ou float
- / | Division réelle | float
- // | Division entière | int
- % | Modulo | int
- ** | Puissance | int ou float

20 Questions intelligentes — Opérations mathématiques en Python

Format : Énoncé → Solution complète avec démarche explicative et démonstration observable

1) Pourquoi $0.1 + 0.2$ ne donne pas exactement 0.3 ?

Énoncé

Expliquer le comportement inattendu des nombres flottants en Python.

Solution

```
"""
Explication
Les nombres float sont représentés en binaire (IEEE 754).
Certaines valeurs décimales comme 0.1 et 0.2 ne peuvent pas être représentées exactement.
Le résultat affiché montre donc l'approximation binaire interne.
"""

print("Addition float classique :")
resultat = 0.1 + 0.2
print(resultat)

print("\nComparaison avec 0.3 :")
print(resultat == 0.3)
```

```

print("\nSolution adaptée pour calculs précis (finance) :")
from decimal import Decimal

a = Decimal("0.1")
b = Decimal("0.2")
print(a + b)

```

2) Pourquoi $4 / 2$ retourne 2.0 et non 2 ?

Énoncé

Expliquer pourquoi la division retourne toujours un float.

Solution

```

"""
Explication
L'opérateur / effectue une division réelle.
Python 3 a unifié le comportement pour toujours retourner un float.
Si on veut un entier, il faut utiliser // ou convertir explicitement.
"""

print(4 / 2)
print(type(4 / 2))

print("Division entière :")
print(4 // 2)
print(type(4 // 2))

```

3) Quelle différence entre $//$ et $\text{int}(a / b)$?

Énoncé

Comparer division entière et conversion.

Solution

```

"""
Explication
// effectue un floor (arrondi vers le bas).
int() tronque vers zéro.
La différence apparaît avec les nombres négatifs.
"""

```

```
print("Division entière :")
print(-10 // 3)

print("Troncature avec int :")
print(int(-10 / 3))
```

4) Pourquoi $-5 // 2$ retourne -3 ?

Énoncé

Expliquer le comportement des divisions entières négatives.

Solution

```
"""
Explication
// effectue un floor mathématique.
-5 / 2 = -2.5
Le floor de -2.5 est -3 (vers le bas).
"""

print(-5 / 2)
print(-5 // 2)
```

5) Relation fondamentale entre $//$ et $\%$

Énoncé

Démontrer la relation mathématique $a = (a // b) * b + (a \% b)$

Solution

```
"""
Explication
Python garantit la cohérence entre division entière et modulo.
On vérifie l'identité mathématique.
"""

a = 17
b = 5

print("Division entière :", a // b)
print("Modulo :", a % b)
```

```
verification = (a // b) * b + (a % b)
print("Recomposition :", verification)
print("Égalité vérifiée :", verification == a)
```

6) Pourquoi `int(3.9)` ne donne pas 4 ?

Énoncé

Comprendre la différence entre troncature et arrondi.

Solution

```
"""
Explication
int() supprime simplement la partie décimale.
Pour arrondir correctement, utiliser round().
"""

print(int(3.9))
print(round(3.9))
```

7) Pourquoi `round(2.5)` donne 2 ?

Énoncé

Expliquer le rounding bancaire.

Solution

```
"""
Explication
Python utilise l'arrondi au pair (Banker's rounding).
2.5 devient 2 car 2 est pair.
3.5 devient 4.
"""

print(round(2.5))
print(round(3.5))
```

8) Pourquoi $10 \% -3$ donne -2 ?

Énoncé

Comprendre le modulo avec nombres négatifs.

Solution

```
"""
Explication
Le résultat du modulo prend le signe du diviseur.
La relation mathématique reste respectée.
"""

print(10 % -3)
print(10 // -3)
```

9) Priorité des opérateurs

Énoncé

Prédire le résultat de $3 + 4 * 2 ** 2$

Solution

```
"""
Explication
Ordre : puissance → multiplication → addition.
2 ** 2 = 4
4 * 4 = 16
3 + 16 = 19
"""

print(3 + 4 * 2 ** 2)
```

10) Pourquoi $1e3$ est valide ?

Énoncé

Expliquer la notation scientifique.

Solution

```
"""
Explication
```

```
1e3 signifie 1 × 10³.  
C'est une notation float.  
"""
```

```
print(1e3)  
print(type(1e3))
```

11) Pourquoi bool est un int ?

Énoncé

Démontrer que `True == 1` et `False == 0`

Solution

```
"""  
Explication  
bool hérite de int.  
True vaut 1.  
False vaut 0.  
"""  
  
print(True + True)  
print(True == 1)  
print(isinstance(True, int))
```

12) Pourquoi `2 ** 1000` fonctionne ?

Énoncé

Expliquer la gestion des grands entiers.

Solution

```
"""  
Explication  
Les int en Python ont une précision arbitraire.  
Ils ne sont pas limités à 32 ou 64 bits.  
"""  
  
print(2 ** 100)
```

13) Pourquoi $0.3 == 0.1 + 0.2$ est faux ?

Énoncé

Montrer une comparaison correcte.

Solution

```
"""
Explication
Comparer des floats nécessite une tolérance.
Utiliser math.isclose().
"""
```

```
import math

a = 0.1 + 0.2
print(a == 0.3)
print(math.isclose(a, 0.3))
```

14) Différence entre $+=$ et $+$?

Énoncé

Expliquer l'opérateur d'affectation augmentée.

Solution

```
"""
Explication
+= modifie la variable existante.
C'est équivalent à  $x = x + \text{valeur}$ .
"""
```

```
x = 5
x += 3
print(x)
```

15) Pourquoi `float("3.5")` fonctionne mais `float("3,5")` non ?

Énoncé

Comprendre le parsing numérique.

Solution

```
"""
Explication
Python utilise le point comme séparateur décimal.
La virgule provoque une erreur.
"""

print(float("3.5"))

try:
    print(float("3,5"))
except ValueError as e:
    print("Erreur :", e)
```

16) Différence entre abs() et math.fabs()

Énoncé

Comparer comportement et type retourné.

Solution

```
"""
Explication
abs conserve le type d'origine.
math.fabs retourne toujours un float.
"""

import math

print(abs(-5))
print(math.fabs(-5))
```

17) Pourquoi pow() existe alors que ** existe ?

Énoncé

Comparer pow et opérateur puissance.

Solution

```
"""
Explication
pow(a, b) est équivalent à a ** b.
```



```
pow(a, b, mod) permet un calcul modulaire optimisé.
"""
```

```
print(pow(2, 3))
print(2 ** 3)
print(pow(2, 5, 3))
```

18) Division avec nombres très grands

Énoncé

Observer la conversion implicite en float.

Solution

```
"""
Explication
La division / convertit en float.
Les très grands nombres peuvent perdre en précision.
"""
```

```
grand = 10**50
print(grand / 3)
```

19) Pourquoi 5 * "3" fonctionne mais 5 + "3" non ?

Énoncé

Expliquer surcharge d'opérateur.

Solution

```
"""
Explication
* répète une chaîne.
+ tente une concaténation incompatible avec int.
"""
```

```
print(5 * "3")

try:
    print(5 + "3")
except TypeError as e:
    print("Erreur :", e)
```

20) Cas métier : calcul de remise

Énoncé

Calculer un total avec remise et TVA en évitant les erreurs float.

Solution

```
"""
Explication
Pour des calculs financiers, utiliser Decimal.
Éviter float pour garantir la précision.
"""

from decimal import Decimal

prix = Decimal("199.99")
remise = Decimal("0.10")
tva = Decimal("0.19")

total = prix * (1 - remise)
total_ttc = total * (1 + tva)

print("Total après remise :", total)
print("Total TTC :", total_ttc)
```